

Architecture Review – Marketplace Backend

Overview

This project is a backend system for a multi-vendor marketplace platform.

The system enables:

- Users to register and authenticate
- Store owners to manage stores and products
- Customers to browse, search, and filter products
- Future support for orders, carts, and wishlists

High-Level Requirements

- Scalable backend architecture
 - Secure authentication system (JWT-based)
 - Efficient product search and filtering
 - Clear separation of concerns in codebase
 - Future-proof design for horizontal scaling and microservices migration
-

System Architecture Diagram

The system follows a **modular monolith architecture** with a layered internal structure.

Components:

- Frontend (external client)
 - Backend (Node.js + Express)
 - Database (SQL via Sequelize ORM)
 - External Services:
 - Redis (rate limiting)
 - Swagger (API documentation)
-

Architecture Decisions

Decision: Modular Monolith Backend

Alternatives:

- Pure Monolith
- Microservices

Chosen:

- Modular Monolith with domain-based modules

Reason:

- Small team and MVP scope
 - Faster development cycle
 - Maintains clear module boundaries
 - Enables future extraction into microservices
-

Decision: Layered Architecture within Modules

Structure:

- Controller
- Service
- Utils (light repository abstraction in MVP stage)

Reason:

- Keeps business logic separated from HTTP concerns
 - Improves readability and maintainability
 - Avoids over-engineering early-stage repository interfaces
-

API Design Strategy

Chosen: REST API

Alternatives:

- GraphQL

Reason:

- Simplicity and industry standard adoption
 - Easier caching and debugging
 - Fits CRUD-heavy marketplace domain
-

Database Strategy

Chosen: Relational SQL database (via Sequelize ORM)

Reason:

- Strong relational structure (users, products, orders)
 - Supports joins for marketplace queries
 - Ensures data consistency
-

Scalability Considerations

- Modular boundaries allow future microservice extraction (auth, products, orders)
- Cursor-based pagination for large datasets
- Indexed database queries for filtering performance
- Redis-based rate limiting layer
- Stateless authentication using JWT

Future improvements:

- Caching layer (Redis for product listings)
 - Search engine integration (Elasticsearch)
 - Horizontal scaling via service decomposition
-

Lessons Learned

- System design must consider **future scaling, not just MVP scope**
 - Modular monoliths provide a strong middle ground between speed and scalability
 - Early architectural discipline prevents costly refactoring later
 - Clear module boundaries are more valuable than premature microservices
-

Tradeoffs

- Avoided heavy repository abstraction in MVP phase
- Accepted some duplication for faster development speed